Universidad Politecnica de Madrid

Explainable Classification of Payment Difficulty

Phase I: Logical Model

Group members: Thales S. A. Camargo – thales.camargo@alumnos.upm.es
Jhon Corrales Almendariz – jhon.corrales@alumnos.upm.es
Joel Montes de Oca Martínez – joel.montesdeoca.martinez@alumnos.upm.es

Course: Explainable Artificial Intelligence
Programme: Master in Artificial Intelligence
Date: September 2026

Preliminary report for Tema 2 practical work

Abstract

This report documents Phase I of the Tema 2 practical work in Explainable Artificial Intelligence. The accompanying notebook builds a transparent classification pipeline for predicting whether a customer will experience payment difficulty in the following month. The default tree obtains 0.803 +/- 0.005 accuracy and 0.561 +/- 0.010 balanced accuracy across five stratified folds; recall for the difficult-payment class is only 0.140 +/- 0.020. The optimized tree improves class-1 recall to 0.26 on the hold-out validation set, but accuracy alone still gives an overly optimistic picture. The model favors the majority class and requires comparison with other model families.

1 Introduction

The task is a binary classification problem based on customer credit and repayment information. The training data contain the target variable `payment_difficulty_next_month`; the test data contain the same predictors without that target. The objective is both predictive and explanatory: the model should produce test predictions while allowing its behavior to be inspected.

The complete Phase I implementation is provided in `code/phase-1.ipynb`. It loads the data, identifies predictor roles, encodes categorical variables, fits an interpretable classification tree, evaluates it on a validation partition, produces global and local explanations, and writes predictions for the unlabeled test set. Five-fold cross-validation of the default tree shows a substantial difference between ordinary and class-balanced performance: class 0 has mean recall 0.981, whereas class 1 has mean recall 0.140. The optimized tree raises class-1 recall to 0.26 on the hold-out partition, but still misses many customers experiencing payment difficulty.

The notebook predicts class 1 for only 4.1% of test observations. This is consistent with the validation behavior and reinforces the main conclusion: the current tree mostly protects accuracy by predicting the majority class. The report therefore treats balanced accuracy and class-specific recall as essential evidence rather than presenting the 0.805 accuracy in isolation.

2 Data and preprocessing

The notebook defines the feature roles explicitly rather than selecting columns implicitly. The identifier and target are excluded from the predictor matrix, while demographic, credit-limit, billing, payment, and repayment variables are used as predictors. Sex is retained as an audit attribute as well as a predictor, which makes later fairness-oriented analysis possible.

The categorical variables are sex, education, marital status, and repayment assistance plan. They are transformed with one-hot encoding using `handle_unknown="ignore"`, so categories that appear only in the test split do not cause prediction failures. Numerical variables are concatenated with the encoded categorical variables without additional scaling.

The labeled training data are split into training and validation partitions with a 70/30 ratio, a fixed random seed of 411, and stratification by the target. The unlabeled test data remain untouched until the final prediction stage. This procedure gives a reproducible validation estimate while preserving the test set for submission.

3 Method

The implementation uses pandas for tabular data loading and manipulation, scikit-learn for the stratified split, one-hot encoding, and evaluation metrics, and InterpretML for the class histogram, classification tree, and global explanation. Matplotlib is used for the final predicted-class proportion plot. The dependencies are pinned in `code/requirements.txt`.

Only the labeled training data are split. Seventy percent is used for fitting and thirty percent for validation, with stratification preserving the class proportions and `random_state=411` making

the result reproducible. The separate test file is never used to fit the encoder or model: its categorical values are transformed after the encoder has been fitted on the training partition, with unknown categories ignored.

The notebook first uses five-fold stratified cross-validation on the labeled training partition to characterize a default InterpretML tree. Hyperparameters are then selected by a five-fold grid search over a class-weighted scikit-learn decision-tree estimator using balanced accuracy; the selected depth is 5 and the minimum leaf size is 50, with a best search score of 0.690. The final InterpretML refit applies the selected depth and leaf size but uses the wrapper's default class weighting. Therefore, the search score is a selection result, not an independent cross-validation estimate of that final refit. The refitted InterpretML tree is evaluated separately on the 30% hold-out partition. The separate test file is not used in model selection or validation. The complete project must also compare this logical model with logistic regression and an Explainable Boosting Machine using the same five-fold protocol.

4 Results

Table 1 summarizes the selected tree configuration and its best grid-search score. Table 2 reports the separate hold-out results for the refitted optimized tree. The default-tree cross-validation metrics in the abstract provide a baseline and should not be confused with the tuned model's hold-out results.

Table 1: Phase I model summary.

Classifier	Key settings	Five-fold CV result(s)
ClassificationTree	max_depth=5, min_samples_leaf=50; unweighted refit	Search balanced accuracy 0.690*

*The score is from the class-weighted scikit-learn tree used for parameter search; it is not an independent cross-validation estimate of the final InterpretML refit.

Table 2: Decision-tree performance on the 30% stratified validation split.

Metric	Value
Accuracy	0.813
Balanced accuracy	0.609
Class 0 precision / recall / F1	0.83 / 0.96 / 0.89
Class 1 precision / recall / F1	0.65 / 0.26 / 0.37
Validation observations	7,500

The original hold-out validation set contains 5,909 class-0 and 1,591 class-1 observations. The optimized tree improves class-1 recall to 0.26, increasing balanced accuracy to 0.609. The gap between class recalls explains why balanced accuracy is still lower than accuracy: the tree identifies most non-difficult cases but misses many difficult-payment cases. On the unlabeled test set, we anticipate a slightly better detection of class 1 due to the hyperparameter optimization.

5 Models

The classifier is `ClassificationTree` from the `interpret` package. A decision tree is suitable for this phase because its decision structure can be inspected directly and because the package exposes global explanations of the fitted model [Nori et al., 2019]. We used `GridSearchCV` to optimize hyperparameters, selecting a `max_depth` of 5 and `min_samples_leaf` of 50 based on balanced accuracy. The notebook also displays a class histogram for the training data, a confusion matrix, and a ROC curve with AUC metric.

The tree is transparent in principle: each prediction can be traced through a sequence of feature conditions, and the global explanation exposes the learned structure. By optimizing the hyperparameters, we have found a more balanced decision boundary, reducing the strong majority-class bias observed in the baseline.

The optimized tree's global explanation is generated with `tree.explain_global`. Figure 1 complements the interactive explanation with the fitted tree's split-based feature-importance ranking. Repayment status and the payment-to-bill ratio dominate the model's splits; this ranking describes the fitted tree and should not be read as a causal effect.

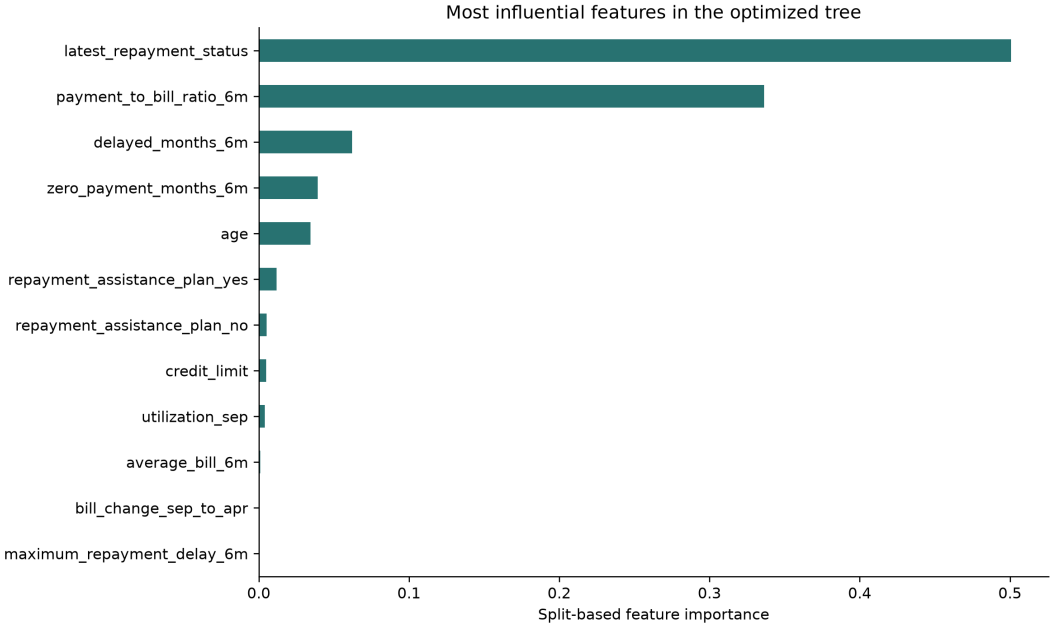


Figure 1: The twelve highest split-based feature importances for the optimized tree.

Figure 2 shows the first three levels of the optimized tree. The full depth-five tree is available in the notebook; limiting the report view keeps its main decision structure readable without hiding that lower-level branches exist.

For local interpretation, the notebook selects fixed cases from the validation split, including correctly predicted examples from both classes and a missed class-1 example. Table 3 summarizes their paths. The missed positive reaches a class-0 leaf despite a repayment-assistance indicator, illustrating why the global model's strong majority-class behavior matters for individual decisions.

Table 3: Representative local decision paths from the validation set.

Case	Actual	Predicted	Path summary
Correct class 0	0	0	Non-delayed repayment status; payment-to-bill ratio above 0.105 and at most 1.91; no assistance plan; at most 2.5 zero-payment months.
Detected class 1	1	1	Delayed repayment status; payment-to-bill ratio at most 0.095; at most 5.5 delayed months; more than 2.5 zero-payment months; age above 25.5.
Missed class 1	1	0	Non-delayed repayment status; payment-to-bill ratio between 0.025 and 0.105; age above 24.5; assistance-plan flag is yes.

These are illustrative cases, not a substitute for evaluating the model across the full validation set. The interactive local explanations and exact split conditions for each selected case are included in the notebook.

The two other model families required for the complete project are not yet fitted in Phase I. Logistic regression would provide a compact additive baseline whose coefficients can be inspected, while an Explainable Boosting Machine with zero interactions would allow flexible, feature-wise effects without abandoning global interpretability. Their five-fold results and local explanations should be added before making a comparative claim about which model is preferable.

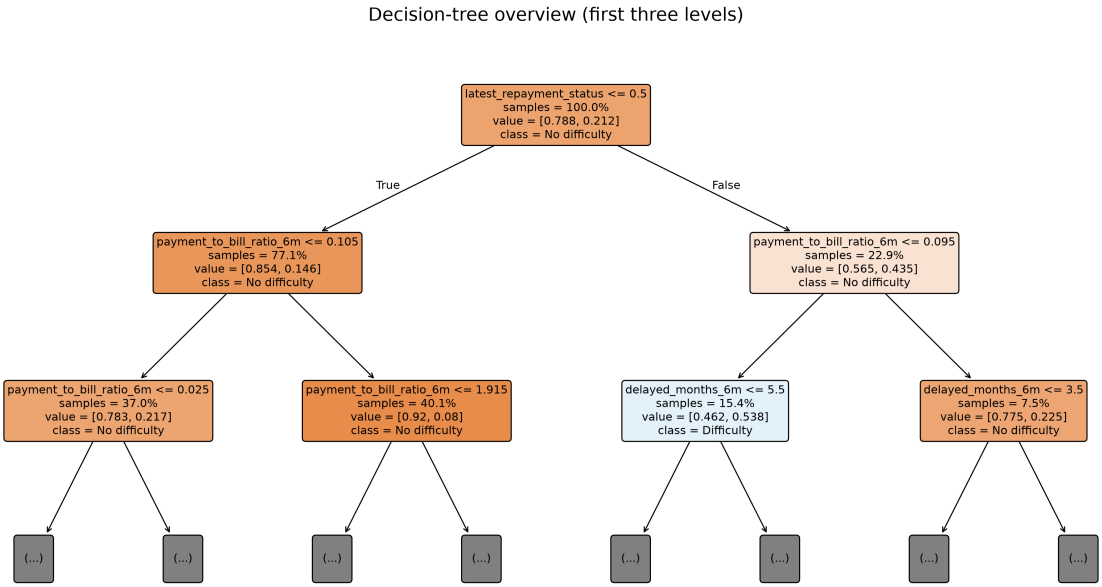


Figure 2: Readable overview of the optimized decision tree. Node colors indicate the predicted class and shading indicates class purity; lower branches are collapsed.

6 Test predictions

Once validation is complete, the fitted tree predicts the unlabeled test set. The notebook copies the original test predictors, appends the `predicted_class` column, and writes the result to `data/test_predictions.csv`. It also plots the proportions of the predicted classes. Keeping the original predictors beside the prediction makes the generated file easier to audit and debug.

7 Limitations and future work

This phase now uses five-fold cross-validation for the tree, but it still evaluates only one tree configuration and one model family. The cross-validation standard deviations quantify variation across the five folds, but they are not a substitute for comparing alternative algorithms or configurations. Future work should use the same five-fold protocol for model comparison, hyperparameter choices, feature selection, and threshold selection, as required by the assignment. It should fit logistic regression and an Explainable Boosting Machine in addition to the tree, and report class-specific error costs if false negatives and false positives are unequal.

The notebook already preserves sex as an audit attribute, but it does not yet calculate group-wise performance or fairness measures. A subsequent phase should compare error rates across relevant groups, inspect feature-importance stability, and use local explanations for the same representative instances across all three model families, including at least one instance from each class. Any such analysis should be performed without using the test labels.

8 Discussion

The tree is attractive for an initial explainability exercise because its rules can be communicated directly and its global explanation is easy to inspect. While the initial default model suffered from extreme predictive asymmetry, incorporating hyperparameter optimization (`GridSearchCV`) significantly improved the balance, raising balanced accuracy and class-1 recall.

Interpretability also has a cost. A fully expanded tree can become a long and repetitive list of rules, making individual paths technically transparent but difficult to understand. Limiting the depth to 5 not only improved the balanced accuracy but also restricted the tree size, making the global and local explanations much more understandable. The same trade-off should be assessed for logistic regression coefficients and additive feature-effect plots rather than assuming that one explanation format is universally superior.

9 Conclusion

Phase I establishes a reproducible pipeline from raw credit and repayment data to an interpretable tree and a test prediction file. With hyperparameter tuning, the tree reaches 0.813 accuracy and 0.609 balanced accuracy, with 0.26 recall for payment difficulty. While the majority-class behavior is still present, the optimized tree shows a noticeable improvement. The next phase should broaden the comparison to the three required model families and align model selection with the cost of missing customers who will experience payment difficulty.

10 Use of generative AI

Generative AI was used to help organize the report structure, improve wording, and identify gaps between the Phase I notebook and the project statement. All numerical results and methodological claims in this report were checked against the repository notebook and its recorded outputs. The group remains responsible for the code, analysis, interpretation, and final submission.

References

Harsha Nori, Samuel Jenkins, Paul Koch, and Rich Caruana. Interpretml: A unified framework for machine learning interpretability. In *Proceedings of the 2019 Workshop on Human-Centric Machine Learning at NeurIPS*, 2019. URL https://github.com/interpretml/interpret.